# সফটওয়্যার ইঞ্জিনিয়ারিং: সমাধানসহ পাস্ট প্রশ্ন

২৫টি বহুনির্বাচনি ও ৩টি বর্ণনামূলক — উত্তর ও ব্যাখ্যাসহ




## ➜ এই নোটে যা আছে

- **SDLC মডেল** — ওয়াটারফল, ভি-মডেল, স্পাইরাল, প্রোটোটাইপ
- **টেস্টিং** — ব্ল্যাক বক্স, হোয়াইট বক্স, সমতুল্য শ্রেণি, সীমানা মান
- **প্রকল্প ব্যবস্থাপনা** — গ্যান্ট চার্ট, COCOMO, ঝুঁকি বিশ্লেষণ
- **ডিজাইন** — কাপলিং, কোহেশন, গ্র্যাস্প প্যাটার্ন, DCD
- **UML** — ইউজ কেস, সিকোয়েন্স, ক্লাস ডায়াগ্রাম
- **২৫টি MCQ ও ৩টি বর্ণনামূলক** — উত্তর ও ব্যাখ্যাসহ

**কীভাবে ব্যবহার করবেন:** প্রথমে পাতা ১০-এর **ভাইভা র‍্যাপিড-ফায়ার** দিয়ে দ্রুত সব প্রশ্ন একবার পড়ুন। তারপর যেসব প্রশ্ন ভালো লাগবে শুধু সেগুলোর ব্যাখ্যা গভীরভাবে পড়ুন। শেষ পাতার অনুশীলনী দিয়ে নিজে চেষ্টা করুন।

পরীক্ষায় প্রায় সবই **MCQ** আকারে আসে। তাই ব্যাখ্যা না পড়েও উত্তর মুখস্থ থাকলে কমপক্ষে ৪-৫ নম্বর নিশ্চিত।

# 1 SDLC মডেল ও প্যারাডাইগম

**MCQ** কোন SDLC model ঝুঁকি-চালিত, যেখানে প্রতিটি loop-এ planning, risk analysis, engineering ও evaluation পুনরাবৃত্তি হয়?

| ক. Waterfall | খ. V-model | গ. **Spiral** | ঘ. Big Bang |
|---|---|---|---|

**উত্তর: গ (C)** — Barry Boehm-এর Spiral model-এ প্রতিটি iteration-এ আলাদাভাবে risk analysis হয়, তাই এটি বড় ও ঝুঁকিপূর্ণ প্রকল্পের জন্য উপযোগী।

**MCQ:** SDLC-এর কোন ধাপে SRS ডকুমেন্ট তৈরি হয়?

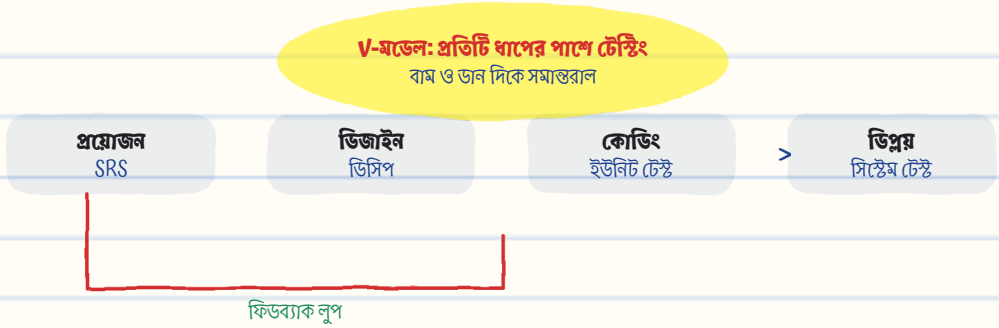
উত্তর: ক — Requirements analysis ও specification। এটিই পরবর্তী design ও testing-এর ভিত্তি।

**MCQ:** কোন model-এ প্রতিটি development ধাপের সাথে একটি testing ধাপ জোড়া থাকে?

উত্তর: খ — V-model। বামে development, ডানে মিল করে testing: requirements↔acceptance, system design↔system, architectural design↔integration, module design↔unit।

**MCQ:** Prototyping model সবচেয়ে উপযোগী কখন?

উত্তর: ঘ — গ্রাহকের requirements অস্পষ্ট এবং যাচাই করে বের করতে হবে।




চারটি মডেল মনে রাখার সংক্ষিপ্ত রূপ: **ওয়াটারফল** — রেখা, **ভি-মডেল** — ভি আকৃতি, **স্পাইরাল** — পুরু লম্বা কুণ্ডলী, **প্রোটোটাইপ** — নমুনা।

## 2 ব্ল্যাক বক্স বনাম হোয়াইট বক্স

| বিষয় | ব্ল্যাক বক্স | হোয়াইট বক্স |
|---|---|---|
| কী দেখে | শুধু স্পেসিফিকেশন | কোডের ভেতরের গঠন |
| কে করে | tester | developer |
| কোন স্তরে | সিস্টেম, অ্যাকসেপ্টেন্স | ইউনিট |
| কৌশল | সমতুল্য শ্রেণি, সীমানা মান | স্টেটমেন্ট, ব্রাঞ্চ, পাথ |

**MCQ:** White-box testing-কে আর কী নামে ডাকা হয়?

উত্তর: ঘ — Glass-box (structural) testing। Behavioural testing হলো black-box-এর আরেক নাম।

✓ নামের অর্থ থেকে মনে রাখুন — **হোয়াইট বক্স** মানে সবাই ভেতরে দেখতে পায়, **ব্ল্যাক বক্স** মানে কোড একটা বাক্সের মতো বাইরে থেকে জোর করে নাড়াতে হয়।

| কৌশল | সুবিধা | সীমাবদ্ধতা |
|---|---|---|
| ব্ল্যাক বক্স | ভুল প্রয়োজনীয়তা ধরে | অপ্রয়োজনীয় কোড পরীক্ষা করে |
| হোয়াইট বক্স | প্রতিটি শর্ত ও পথ যাচাই হয় | প্রয়োজনীয়তার ভুল ধরতে পারে না |

**কেন দুটোই দরকার:** কেবল ব্ল্যাক বক্স করলে কোডের ভেতরের ভুল ফাঁকি দেয়, কেবল হোয়াইট বক্স করলে ভুল প্রয়োজনীয়তা ফাঁকি দেয়। তাই ইউনিট স্তরে হোয়াইট বক্স, সিস্টেম স্তরে ব্ল্যাক বক্স — **দুটো একসাথে**। ★

## 3 হোয়াইট বক্স = কাঠামোগত টেস্টিং

**MCQ** নিচের function-টির cyclomatic complexity V(G) কত?

```
int f(int a,int b,int r){
  if(a>0) return 1;
  while(b>0) b--;
  if(r>10) return 2;
  return 0;
}
```

| ক. 2 | খ. 3 | গ. 4 | ঘ. 5 |
|---|---|---|---|

**উত্তর: গ (C)** — V(G) = সিদ্ধান্ত বিন্দুর সংখ্যা + ১। এখানে তিনটি decision (if, while, if), তাই V(G) = ৩ + ১ = ৪টি স্বাধীন পথ।

V(G) গণনার সূত্র: **সিদ্ধান্ত-বিন্দু + ১**। মনে রাখতে সহজ করে বলা যায় — "যত বেশি শর্ত, তত বেশি পথ, তত জটিল কোড"। বড় V(G) মানে ফাংশনটি ছোট ছোট ভাগ করা উচিত। ★

# 4 সমতুল্য শ্রেণি ও সীমানা মান বিশ্লেষণ

**বর্ণনা** বয়সের ঘর ১৮ থেকে ৬০ (উভয়সহ) বছর নেয়। সমতুল্য পার্টিশনিং ও বাউন্ডারি ভ্যালু বিশ্লেষণ দিয়ে টেস্ট কেস বানান।

| পদ্ধতি | ভাবনা |
|---|---|
| **সমতুল্য শ্রেণি** | সমান আচরণের গ্রুপ বানিয়ে প্রতিটি থেকে একটি মান নেওয়া |
| **বাউন্ডারি মান** | শ্রেণির **প্রান্তে** ও তার পাশের মান নেওয়া |

| শ্রেণি | পরিসর | নমুনা |
|---|---|---|
| অবৈধ | বয়স < ১৮ | ১০ |
| **বৈধ** | ১৮ – ৬০ | ৩৫ |
| অবৈধ | বয়স > ৬০ | ৭০ |

**বাউন্ডারি মান:** ১৭, ১৮, ১৯ এবং ৫৯, ৬০, ৬১।

**প্রত্যাশিত ফল:** ১৭ ও ৬১ প্রত্যাখ্যাত; ১৮, ১৯, ৫৯ ও ৬০ গৃহীত।

↙ পার্থক্যটা ধরুন – সমতুল্য শ্রেণি **গোড়া** মান খোঁজে, বাউন্ডারি মান **প্রান্তের** মান খোঁজে। দুটো একসাথে ব্যবহার করলে কোনো ত্রুটিই ফাঁকি দিতে পারে না।

পরীক্ষায় "১৮ থেকে ৬০" উদাহরণ এলে ছয়টি সংখ্যা লিখতে হবে: **১৭, ১৮, ১৯, ৫৯, ৬০, ৬১**। মাত্র তিনটি (১০, ৩৫, ৭০) লিখলে ভুল ধরা পড়বে। ★

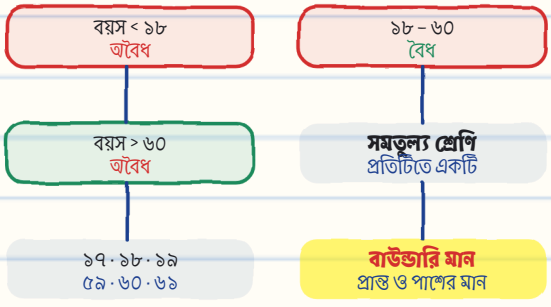
## ভ্যালিডেশন ও ভেরিফিকেশন

**ভেরিফিকেশন** — "আমরা কি সঠিকভাবে তৈরি করছি?" অভ্যন্তরীণ। **ভ্যালিডেশন** — "আমরা কি সঠিক জিনিস তৈরি করছি?" SRS-এর সাথে তুলনা করে, গ্রাহক জড়িত। দুটোকে একসাথে বলে **ভি ও ভি (V&V)**।

↙ CMMI-তে **সংজ্ঞায়িত প্রক্রিয়া** হলো স্তর ৩। স্তর ১ প্রাথমিক, ২ পরিচালিত, ৪ পরিমাপিত, ৫ অপ্টিমাইজড।

**আরও তিনটি কৌশল:** *Decision table* — শর্তের সব সংমিশ্রণ থেকে ফলাফল। *State transition* — অবস্থা বদলের নিয়ম পরীক্ষা। *Use case testing* — গ্রাহকের প্রতিটি কাজ অনুসারে পরীক্ষা।

কোনটা কখন — **ইনপুটের মান** বেছে নিতে সমতুল্য শ্রেণি; **শর্তের সংমিশ্রণ** বেছে নিতে ডিসিশন টেবিল। ★



## 5 সফটওয়্যার ডেভেলপমেন্ট লাইফ সাইকেল

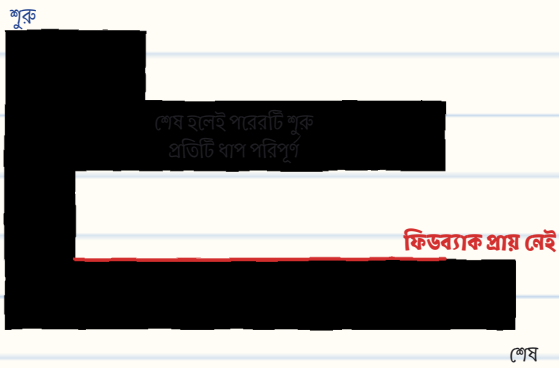
**বর্ণনা** *Waterfall model* ও এর ধাপগুলো বর্ণনা করো। দুটি সুবিধা ও দুটি অসুবিধা লেখো।

**মডেল:** এটি একটি *linear-sequential SDLC model* — আগের ধাপ সম্পূর্ণ ও অনুমোদিত হওয়ার পরই পরের ধাপ শুরু হয়, এবং এক ধাপের *output* পরের ধাপের *input* হয়।

| | |
|---|---|
| ১ | *Requirements analysis* ও *specification* (*SRS* তৈরি) |
| ২ | *System* ও *software design* |
| ৩ | *Implementation* (*coding*) ও *unit testing* |
| ৪ | *Integration* ও *system testing* |
| ৫ | *Deployment* |
| ৬ | *Maintenance* |

**সুবিধা:** (ক) বোঝা ও পরিচালনা করা সহজ — প্রতিটি ধাপের স্পষ্ট *milestone* ও *deliverable* থাকে। (খ) *requirements* স্থির ও পরিষ্কার ছোট প্রকল্পে ভালো কাজ করে এবং ভালো *documentation* তৈরি হয়।

**অসুবিধা:** (ক) *requirements* পরিবর্তন ব্যয়বহুল, কারণ আগের ধাপে ফিরে যাওয়া কঠিন। (খ) কার্যকর *software* অনেক দেরিতে পাওয়া যায়, তাই ভুল দেরিতে ধরা পড়ে — বড় বা অনিশ্চিত প্রকল্পে ঝুঁকিপূর্ণ।




**ওয়াটারফল: ধাপে ধাপে নামা**

## 6 ওয়াটারফল বনাম ইনক্রিমেন্টাল

| বিষয় | ওয়াটারফল | ইনক্রিমেন্টাল |
|---|---|---|
| ডেলিভারি | শেষে একবার সম্পূর্ণ | কয়েকবার আংশিক |
| ফিডব্যাক | দেরিতে | প্রতি ধাপে |
| উপযোগী | স্থির প্রয়োজনা, ছোট প্রকল্প | পরিবর্তনশীল প্রয়োজনা |
| ঝুঁকি | উচ্চ | কম |

## 7 প্রকল্প সময়সূচি ও গ্যান্ট চার্ট

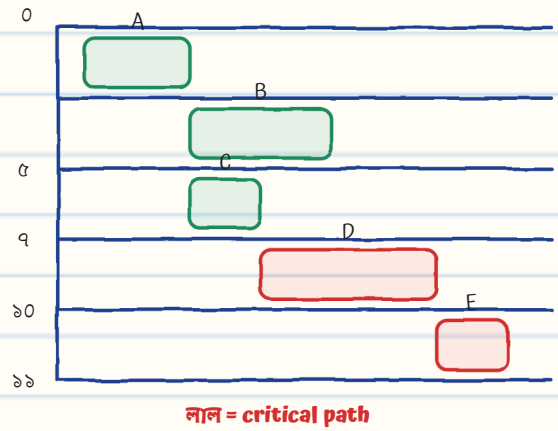
**MCQ** কাজ: A(৩ দিন, কোনো পূর্বসূরি নেই), B(৪ দিন, A-র পরে), C(২ দিন, A-র পরে), D(৫ দিন, C-র পরে), E(১ দিন, B ও D-এর পরে)। B-এর total float কত?

**উত্তর: ঘ (D) — ৩ দিন**

A–B–E পথ = ৩+৪+১ = ৮ দিন; A–C–D–E পথ = ৩+২+৫+১ = ১১ দিন। তাই critical path = A–C–D–E, মোট ১১ দিন।

E দিন ১০-এর মধ্যে শুরু হতে হবে, তাই B-এর latest finish = ১০। B-এর earliest finish = ৩+৪ = ৭।

**Float = ১০ − ৭ = ৩ দিন।**



লাল = critical path

**গ্যান্ট চার্ট:** অনুভূমিক অক্ষে সময়, উল্লম্বে কাজ। প্রতিটি কাজের জন্য একটি বার লেখা হয় যার শুরু ও শেষ সময় দেখায়। সুবিধা — সময়সূচি সহজে বোঝা যায়; অসুবিধা — নির্ভরশীলতা ও critical path স্পষ্ট নয়।

↙ Critical path-এর কোনো কাজের float **শূন্য**। এটি সবচেয়ে দীর্ঘ পথ, তাই এটি বিলম্বিত হলে পুরো প্রকল্প দেরি হয়। ★

**কাজের তিন সময়:** Earliest Start (ES) — যত তাড়াতাড়ি শুরু করা যায়। Latest Finish (LF) — যত দেরি করে শেষ করা যায় প্রকল্পের সময় না ভেঙে। Total Float = LF − earliest finish। ফ্লোট শূন্য হলে সেই কাজ critical path-তে।

↙ সাবজেক্টিভ মানে পরিমাপযোগ্য ফলাফল দেওয়া, এবং তার সাথে অনুমান না থাকা। "প্রকল্প ৯০% সম্পন্ন" বলা ভালো নয়; "প্রতিটি মডিউলের ইউনিট টেস্ট শেষ, সেটআপ ৮/১০ সম্পন্ন" – এটাই স্মার্ট কীওয়ার্ক।

## 8 CI/CD

**CI (Continuous Integration)** — ডেভেলপার প্রতিদিন কোড মার্জ করেন এবং স্বয়ংক্রিয় টেস্ট চলে। **CD (Continuous Delivery/Deployment)** — পরীক্ষা পাশ করা কোড স্বয়ংক্রিয়ভাবে প্রকাশ হয়।

সুবিধা তিনটি: **দ্রুত বাগ শনাক্ত, ছোট ছোট কমিট** তাই সহজে পুনরায় করা যায়, এবং **প্রতি মুহূর্তে কাজ করা অবস্থা** পাওয়া যায়। ★

## 9 COCOMO: উন্নয়নের সময়

**MCQ** Basic COCOMO-র কোনটি project mode নয়?

| | | | |
|---|---|---|---|
| ক. Organic | খ. Semi-detached | গ. Distributed | ঘ. Embedded |

**উত্তর: গ (C) — Distributed**

Boehm-এর Basic COCOMO প্রকল্পকে organic, semi-detached ও embedded — এই তিনটি mode-এ ভাগ করে।

**MCQ** Organic প্রকল্পে Effort = 2.4 × (KLOC)$^{1.05}$ person-month। 10 KLOC হলে effort কত?

**উত্তর: খ (B) — প্রায় ২৭**

$10^{1.05}$ = ১০ × ১.১২২ ≈ ১১.২২। Effort = ২.৪ × ১১.২২ ≈ ২৬.৯ ≈ ২৭ person-month।

↙ Basic COCOMO-র প্রথমে KLOC হিসাব হয়, তারপর effort, তারপর schedule ও তারপর cost পাওয়া যায়। মনে রাখুন: **আকার → সম্পদ → সময় → খরচ**।

## 10 ঝুঁকি বিশ্লেষণের কৌশল

| কৌশল | কী করে |
|---|---|
| ঝুঁকি সনাক্তকরণ | সম্ভাব্য ঝুঁকি তালিকাভুক্ত করে |
| **ঝুঁকি মূল্যায়ন** | সম্ভাবনা × ক্ষতি = exposure বের করে |
| ঝুঁকি নিয়ন্ত্রণ | ক্ষতি কমাতে পদক্ষেপ নেয় |
| ঝুঁকি পর্যবেক্ষণ | পরিবর্তন ট্র্যাক করে |

**MCQ** ঝুঁকির সম্ভাবনা ০.৩, ক্ষতি হলে ৫০,০০০ টাকা। Risk exposure কত?

**উত্তর: ক (A) — ১৫,০০০ টাকা**

**RE = P × C = ০.৩ × ৫০,০০০ = ১৫,০০০ টাকা।**

ক্যাটাগরির ভিত্তিতে: সম্ভাবনা ও ক্ষতি দুটোই বেশি হলে **উচ্চ ঝুঁকি**, কেবল একটি বেশি হলে **মাঝারি**, দুটোই কম হলে **নিম্ন ঝুঁকি**। সবচেয়ে বেশি exposure-এর ঝুঁকিকে আগে সামলাতে হয়। ★

**ঝুঁকি সামলানোর চার কৌশল:** এড়িয়ে যাওয়া (ঝুঁকি এড়ানোই সস্তা), হস্তান্তর (বীমা কিনে ঝুঁকি অন্যকে দেওয়া), ঝুঁকি কমানো (আরও পরীক্ষা করে নিশ্চয়তা বাড়ানো), এবং সহ্য করা (বাজেট রেখে দেওয়া)।

↙ ঝুঁকি সনাক্ত করার কৌশল: ব্রেইনস্টর্মিং, চেকলিস্ট, বিশিষ্টজন পর্যালোচনা, এবং পূর্ববর্তী প্রকল্পের পাঠ। গত প্রকল্পে যা ভুল হয়েছিল, এবার সেটাই সবচেয়ে বাস্তবসম্মত ঝুঁকি।

# 11 কাপলিং ও কোহেশন

**কোহেশন** — একটি *module*-এর ভেতরের *element*-গুলো কতটা একসাথে সম্পর্কিত ও একটি উদ্দেশ্যে কাজ করে। **কাপলিং** — বিভিন্ন *module*-এর পারস্পরিক নির্ভরশীলতার মাত্রা।

↙ ভালো ডিজাইনে উচ্চ কোহেশন ও নিম্ন কাপলিং চাওয়া হয় — তাহলে *module* বোঝা, পরীক্ষা ও পুনঃব্যবহার সহজ হয়, একটির পরিবর্তন অন্যগুলোর উপর কম প্রভাব ফেলে, এবং ত্রুটি এক জায়গায় সীমাবদ্ধ থাকে।

## → কাপলিং — ভালো থেকে খারাপ

| | |
|---|---|
| ১. **Data** | শুধু *parameter*-এ সাধারণ *data* |
| ২. *Stamp* | গঠনটি সাধারণ *(record)* |
| ৩. *Control* | নিয়ন্ত্রণের পতাকা পাঠায় |
| ৪. *External* | বাইরের ফরম্যাটে ভাগ করে |
| ৫. *Common* | একই গ্লোবাল ডেটা |
| ৬. *Content* | সরাসরি অন্যের অভ্যন্তরণ বদলায় |

**MCQ:** কোন *coupling* সবচেয়ে কাঙ্ক্ষিত?

উত্তর: ক — *Data coupling*।

## → কোহেশন — ভালো থেকে খারাপ

| | |
|---|---|
| ১. **Functional** | সব *element* একটি কাজে |
| ২. *Sequential* | একটির *output* আরেকের *input* |
| ৩. *Communicational* | একই ডেটা নিয়ে কাজ করে |
| ৪. *Procedural* | নির্দিষ্ট ক্রমে কাজ করে |
| ৫. *Temporal* | সময়-সম্পর্কিত কাজ একসাথে |
| ৬. *Logical* | অনুরূপ কিন্তু ভিন্ন কাজ |
| ৭. *Coincidental* | কোনো যৌক্তিক সম্পর্ক নেই |

**MCQ:** কোন *cohesion* সবচেয়ে শক্তিশালী, যেখানে *module*-এর প্রতিটি *element* একটি মাত্র সুনির্দিষ্ট কাজে অবদান রাখে?

উত্তর: ক — *Functional cohesion*।

দুটোর মধ্যে পার্থক্য ধরুন — কোহেশন **ভেতরের** *(module*-এর ভিতরে*)*, কাপলিং **বাইরের** *(module*-গুলোর মধ্যে*)*। "ভেতরে একত্রে, বাইরে আলাদা" — এই এক বাক্যেই মনে রাখা যায়। ★

**উদাহরণ:** একটি *Order class*-এর ভেতরে *item* যোগ করা, মোট হিসাব ও ছাপা — তিনটিই একই কাজের অংশ, তাই উচ্চ কোহেশন। কিন্তু *Order* যদি সরাসরি *PaymentGateway*-এর ভেতরের ফিল্ড ছুঁয়ে দাম বানায়, তবে কাপলিং বেড়ে যায়।

↙ কোড স্নিপেট দেখে কাপলিং চিনুন: অন্য *module*-এর ভেরিয়েবলে সরাসরি লেখা = সাধারণ কাপলিং; শুধু প্যারামিটার পাঠানো = ডেটা কাপলিং।

# 12 গ্র্যাস্প প্যাটার্ন ও ডিজাইন ক্লাস ডায়াগ্রাম

| প্যাটার্ন | ভাবনা |
|---|---|
| **Information Expert** | যার কাছে তথ্য আছে, দায়িত্ব তার |
| **Creator** | যে class অন্যকে ধারণ করে, তাই তৈরি করে |
| **Controller** | UI-এর পরের প্রথম object, ইভেন্ট গ্রহণ করে |
| Pure Fabrication | কোনো domain তথ্য নেই, শুধু সমন্বয় |
| Indirection | মাঝের object দিয়ে হিসাব লুকায় |
| Low Coupling | নির্ভরতা কম রাখে |
| Polymorphism | ভিন্ন আচরণ বদলে বসায় |

**MCQ:** যে class-এর কাছে কাজের প্রয়োজনীয় তথ্য আছে, দায়িত্ব সেই class-কে দেওয়া কোন প্যাটার্ন?

উত্তর: গ — Information Expert। যেমন Sale তার line item জানে, তাই মোট হিসাব Sale-এর।

**MCQ:** POS সিস্টেমে Sale অনেক SalesLineItem ধারণ করে। কোন class instance তৈরি করবে?

উত্তর: ক — Sale। যে B, A-কে ধারণ করে, B-ই A তৈরি করে।

**MCQ:** কোন প্যাটার্ন system event (যেমন enterItem) গ্রহণের দায়িত্ব non-UI class-কে দেয়?

উত্তর: খ — Controller। এটি UI layer-এর পরের প্রথম object; facade বা use case (session) controller হতে পারে।

**MCQ:** কোনটি সাধারণত DCD-তে থাকে কিন্তু domain model-এ থাকে না?

উত্তর: গ — Method (operation) ও navigability তীর। Domain model-এ class, attribute, association ও multiplicity থাকে কিন্তু operation থাকে না।

**DCD বনাম Domain model:** Domain model একটি conceptual view — class, attribute ও relationship দেখায়। DCD হলো software view — method, parameter, return type ও navigability যোগ করা হয়।

গ্র্যাস্প মনে রাখতে: **Information Expert, Creator, Controller, Pure Fabrication, Indirection, Low Coupling, Polymorphism** — প্রথম অক্ষরে I C C P I L P। পরীক্ষায় প্যাটার্নের সংজ্ঞা ও উদাহরণ জোড়া মিলিয়ে লিখতে হয়, শুধু নাম যথেষ্ট নয়। ★

# 13 UML ও ইউজ কেস ডায়াগ্রাম

**MCQ:** কোন UML diagram lifeline ও message ব্যবহার করে object-এর ক্রিয়া সময়ের ক্রমানুসারে দেখায়?

উত্তর: খ — Sequence diagram।

**MCQ:** Use case diagram-এ সিস্টেমের সাথে যোগাযোগকারী ব্যক্তি বা বাহ্যিক সিস্টেমকে কী বলা হয়?

উত্তর: গ — Actor। সাধারণত stick figure দিয়ে আঁকা হয়।

**MCQ:** UP-এর কোন phase-এ business case, scope ও মোটামুটি estimate নির্ধারণ হয়?

উত্তর: ক — Inception। Elaboration-এ architecture, Construction-এ বাকি সিস্টেম, Transition-এ deployment।

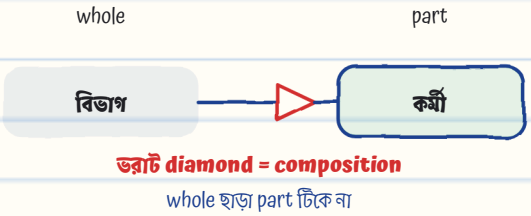
**MCQ:** 'Withdraw Cash' সবসময় 'Authenticate User' করে — এটি কোন সম্পর্ক?

উত্তর: ঘ — <<include>>। এটি বাধ্যতামূলক। <<extend>> শুধু নির্দিষ্ট শর্তে ঐচ্ছিক আচরণ যোগ করে।

**MCQ:** Composition (whole ছাড়া part টিকে না) কীভাবে আঁকা হয়?

উত্তর: খ — Whole-এর প্রান্তে **ভরাট** diamond।

| চিহ্ন | অর্থ |
|---|---|
| ভরাট diamond | Composition — part বাঁধা |
| ফাঁপা diamond | Aggregation — part স্বাধীনভাবেও থাকে |
| ফাঁপা ত্রিভুজ | Generalization (inheritance) |
| Dashed তীর | Dependency |




**ভরাট diamond = composition**
whole ছাড়া part টিকে না

## 14 ভাইভা র‍্যাপিড-ফায়ার

- ✓ **SDLC-এর ছয় ধাপ?** — প্রয়োজন, ডিজাইন, কোডিং, টেস্টিং, ডিপ্লয়, মেইনটেন্যান্স
- ✓ **SRS কী?** — সফটওয়্যার রিকোয়ারমেন্ট স্পেসিফিকেশন, উন্নয়নের ভিত্তি
- ✓ **ভি-মডেলের মূল ভাবনা?** — প্রতিটি *development* ধাপের পাশে যুক্ত *testing* ধাপ
- ✓ **ব্ল্যাক বক্স কৌশল তিনটি?** — সমতুল্য শ্রেণি, বাউন্ডারি মান, ডিসিশন টেবিল
- ✓ **V(G) সূত্র?** — সিদ্ধান্ত-বিন্দু + ১
- ✓ **কাপলিং ও কোহেশনের পার্থক্য?** — বাইরের নির্ভরতা বনাম ভেতরের একত্রতা
- ✓ **Critical path কী?** — সবচেয়ে দীর্ঘ পথ, *float* শূন্য
- ✓ **Risk exposure সূত্র?** — সম্ভাবনা × ক্ষতি
- ✓ **Information Expert কী?** — তথ্য যার কাছে, দায়িত্ব তার
- ✓ **Actor কী?** — সিস্টেমের বাইরের ব্যবহারকারীর ভূমিকা
- ✓ ***include* বনাম *extend*?** — বাধ্যতামূলক বনাম ঐচ্ছিক
- ✓ **Composition চিহ্ন?** — ভরাট *diamond*
- ✓ **মেইনটেন্যান্স কয় প্রকার?** — করেক্টিভ, অ্যাডাপটিভ, পারফেক্টিভ, প্রিভেন্টিভ
- ✓ **সিস্টেম টেস্টিংয়ের চার প্রকার?** — লোড, স্ট্রেস, সিকিউরিটি, কম্প্যাটিবিলিটি
- ✓ **CMMI স্তর ৩ কী?** — সংজ্ঞায়িত প্রক্রিয়া
- ✓ **MTBF ও MTTR?** — ব্যর্থ হওয়ার গড় ব্যবধান ও ঠিক করার গড় সময়

## 15 সব MCQ এক নজরে

| # | প্রশ্নের বিষয় | উত্তর | # | প্রশ্নের বিষয় | উত্তর |
|---|---|---|---|---|---|
| ১ | ঝুঁকি-চালিত মডেল | গ *Spiral* | ১৪ | *Actor* কী | গ |
| ২ | *SRS* তৈরির ধাপ | ক | ১৫ | *UP*-এ *Inception* | ক |
| ৩ | ডেভ ও টেস্ট জোড়া | খ *V-model* | ১৬ | বাধ্যতামূলক সম্পর্ক | ঘ *include* |
| ৪ | *Prototyping* কখন | ঘ | ১৭ | *Composition* চিহ্ন | খ ভরাট |
| ৫ | *COCOMO mode* নয় | গ *Distributed* | ১৮ | *Information Expert* | গ |
| ৬ | *COCOMO effort* | খ ২৭ | ১৯ | *Creator* কে | ক *Sale* |
| ৭ | *B*-এর *float* | ঘ ৩ দিন | ২০ | *Event* গ্রহণ | খ *Controller* |
| ৮ | ভাষা-নিরপেক্ষ আকার | খ *FP* | ২১ | *DCD*-তে অতিরিক্ত | গ *Method* |
| ৯ | *Risk exposure* | ক ১৫,০০০ | ২২ | *White-box* নাম | ঘ *Glass-box* |
| ১০ | সবচেয়ে ভালো *coupling* | ক *Data* | ২৩ | *Cyclomatic V(G)* | গ ৪ |
| ১১ | সবচেয়ে শক্তিশালী *cohesion* | ক *Functional* | ২৪ | নতুন *OS*-এ চলা | খ *Adaptive* |
| ১২ | *Context diagram* | ঘ | ২৫ | *Top-down*-এ স্থানীয় বিকল্প | ঘ *Stub* |
| ১৩ | *Lifeline* ও *message* | খ *Sequence* | | | |

## ✓ অনুশীলনী MCQ (পরীক্ষার ধরন, ১০ নম্বর)

১. কোন SDLC model-এ প্রতিটি loop-এ পৃথক risk analysis হয়?
ক. Waterfall
খ. V-model
গ. Spiral
ঘ. Big Bang

২. SRS ডকুমেন্ট তৈরি হয় কোন ধাপে?
ক. Requirements analysis
খ. Design
গ. Coding
ঘ. Maintenance

৩. কোন মডেলে প্রতিটি development ধাপের সাথে testing ধাপ জোড়া থাকে?
ক. Incremental
খ. V-model
গ. Prototyping
ঘ. RAD

৪. বয়সের ঘর ১৮–৬০ হলে বাউন্ডারি মান কোনগুলো?
ক. ১০, ৩৫, ৭০
খ. ১৭, ১৮, ৬০
গ. ১৭, ১৮, ১৯, ৫৯, ৬০, ৬১
ঘ. ১৮, ২০, ৪০

৫. কোন coupling সবচেয়ে কাঙ্ক্ষিত?
ক. Data
খ. Content
গ. Common
ঘ. Control

৬. কোন cohesion সবচেয়ে শক্তিশালী?
ক. Functional
খ. Temporal
গ. Logical
ঘ. Coincidental

৭. সম্ভাবনা ০.৩, ক্ষতি ৫০,০০০ — risk exposure কত?
ক. ১৫,০০০
খ. ৩৫,০০০
গ. ৫০,০০০
ঘ. ১৫০,০০০

৮. কোন UML diagram-এ lifeline ও message থাকে?